

Migrating an offline-first PWA from Flask to Nostr relays

The migration is technically feasible but hinges on one critical constraint: relay event size limits. Most public relays enforce a 64 KB maximum event size, which means encrypted chart blobs of 100 KB–2 MB cannot be stored inline without compression or architectural changes. The viable path combines JSON compression (fflate, ~82% reduction) with base64 encoding for charts under ~200 KB raw JSON, Blossom blob storage for larger charts, and a self-hosted strfry relay configured for 2 MB events as a persistence guarantee. Every other aspect of the migration — key derivation, signing, real-time sync, sharing, deletion — has mature tooling and proven patterns today.

The Nostr ecosystem's JavaScript stack centers on **nostr-tools v2.23.0** (~20–25 KB gzipped) ([npm](#)) and the **@noble/curves** cryptography suite (audited, ~10 KB gzipped for secp256k1). NIP-06 provides deterministic keypair derivation from any existing BIP39 mnemonic ([Medium](#) [A Security Site](#)) at path `m/44'/1237'/0'/0/0`. ([GitHub](#)) ([E2Encrypted](#)) Schnorr signing takes ~1–3 ms in the browser — imperceptible for per-save signing. The rest of this report covers each of the 12 research areas in depth with current-state implementations, code examples, and specific gotchas.

1. secp256k1 runs fast enough in pure JavaScript

Use **@noble/curves v2.x** — it is independently audited (6 audits including Cure53), ([Paulmillr](#)) already a transitive dependency of nostr-tools, and includes Schnorr (BIP-340) signing via `import { schnorr } from '@noble/curves/secp256k1'`. The single-curve build is **~22 KB minified / ~10 KB gzipped**. ([UNPKG](#)) The older **@noble/secp256k1** package is not deprecated but is a "5 KB sister project" with fewer features ([GitHub](#)) and no independent audit; ([GitHub](#)) ([npm](#)) since nostr-tools depends on **@noble/curves**, using it avoids duplicate dependencies.

NIP-06 standardizes BIP39 → Nostr derivation at path `m/44'/1237'/0'/0/0` ([GitHub](#)) (coin type 1237 is Nostr's SLIP-44 entry). Yes, you can derive a Nostr keypair deterministically from an existing 12-word mnemonic without changing it. Required libraries: **@scure/bip39**, **@scure/bip32**, **@noble/curves**. ([GitHub](#)) The BIP39 English wordlist adds ~100 KB (tree-shakeable).

typescript

```
import { mnemonicToSeedSync } from '@scure/bip39';
import { HDKey } from '@scure/bip32';
import { schnorr } from '@noble/curves/secp256k1';
import { bytesToHex } from '@noble/hashes/utils';

const seed = mnemonicToSeedSync(mnemonic, "");
const child = HDKey.fromMasterSeed(seed).derive("m/44'/1237'/0'/0/0");
const secretKey = child.privateKey; // Uint8Array(32)
const publicKey = schnorr.getPublicKey(secretKey); // Uint8Array(32) x-only
```

Performance is a non-issue. Schnorr signing runs at ~685 ops/sec in Node.js (~1.5 ms/op), [npm](#) translating to roughly **1–3 ms per sign in a browser**. Web Crypto ECDSA-P256 is ~100x faster (native C), but Nostr mandates secp256k1 Schnorr, and Web Crypto does not support secp256k1 at all. Even signing 10 events per save totals 10–30 ms — well below any perceptible threshold. The secp256k1 precompute (~14 ms) runs once at startup.

AES-256-GCM on Web Crypto alongside @noble signing works perfectly. Both APIs operate on [Uint8Array](#)/[ArrayBuffer](#) with zero interop issues. Keeping AES on Web Crypto is strongly recommended: native speed with hardware AES-NI acceleration, constant-time execution resistant to timing attacks, [CryptoKey](#) objects with [extractable: false](#) for key isolation, and zero bundle cost. The architecture is: signing layer = [@noble/curves](#) (JS), encryption layer = Web Crypto API (native), hashing layer = [@noble/hashes](#) (JS for Nostr event IDs).

2. nostr-tools is the ecosystem standard

nostr-tools v2.23.0 ([npm](#): [nostr-tools](#), also published as [@nostr/tools](#) on JSR) is the de facto standard [npm](#) with 309 npm dependents. It's low-level, tree-shakeable via modular imports ([nostr-tools/pure](#), [nostr-tools/pool](#), [nostr-tools/nip44](#)), and depends only on [@noble](#)/[@scure](#) packages. [npm](#) **NDK** ([@nostr-dev-kit/ndk](#) v2.14.33) is higher-level with built-in caching, outbox model, and framework bindings [GitHub](#) — but significantly larger (~80–120+ KB gzipped). For a size-conscious PWA wanting full control, nostr-tools is the right choice.

SimplePool handles relay connection management. Configure with [{ enableReconnect: true, enablePing: true }](#) for exponential backoff reconnection and WebSocket keepalive. [GitHub](#) On reconnect, existing subscriptions are automatically restarted with [since:](#) set to the timestamp of the last received event +1, preventing duplicates and gaps. [GitHub](#) [npm](#)

typescript

```
import { SimplePool } from 'nostr-tools/pool';
const pool = new SimplePool({ enableReconnect: true, enablePing: true });
const sub = pool.subscribeMany(relays, [{ kinds: [30078], authors: [pk] }], {
  onevent(event) { handleIncomingEvent(event); },
  oneose() { console.log('Initial sync complete, now real-time'); }
});
```

NIP-33 works naturally but nostr-tools does not auto-deduplicate replaceable events — you receive raw events and must keep only the highest [created_at](#) per [\(kind, pubkey, d-tag\)](#) tuple client-side. **NIP-44 encryption is built in** [GitHub](#) via [nostr-tools/nip44](#) [GitHub](#) with [getConversationKey](#), [encrypt](#), [decrypt](#) exports [GitHub](#) using the audited v2 spec (ChaCha20 + HMAC-SHA256 + HKDF). [Learnnostr](#) [GitHub](#) **NIP-09 deletion** is trivial: construct a kind 5 event with [a](#) and [k](#) tags. **Bundle size** for typical PWA usage (pure + pool + nip44 + nip19) is approximately **20–25 KB gzipped** — a one-time service worker cache cost.

3. The 64 KB relay limit is the critical constraint

Most public relays running strfry enforce a 64 KB maximum event size. This is the `maxEventSize` default in strfry, the dominant relay software. `githubusercontent` Relay-specific limits:

Relay	Estimated limit	Software
relay.damus.io	~64 KB	strfry
nos.lol	~64 KB	strfry
relay.nostr.band	~64 KB	Custom/strfry
relay.primal.net	Unknown (possibly higher)	Primal custom
purplepag.es	~64 KB (profile-focused)	Specialized

NIP-95 (inline binary storage) was never merged. It was superseded by Blossom and NIP-96. Blossom ("Blobs Stored Simply on Mediaservers") is the dominant solution for large content today — HTTP endpoints for binary upload/download, content-addressed by SHA-256, authenticated via Nostr keys. `GitHub` `Nostrify` Active Blossom servers include blossom.primal.net, blossom.nostr.build (100 MiB max, 50 MiB free), blossom.band, and cdn.satellite.earth. Client libraries: `blossom-client-sdk`, `@nostr-dev-kit/ndk-blossom`. `npm` No standardized NIP exists for chunking events.

The practical architecture is a three-tier approach based on chart size:

For charts $\leq$ ~200 KB raw JSON, the pipeline JSON → compress → encrypt → base64 → event content fits within 64 KB. A 150 KB JSON chart compresses to ~27 KB with fflate (82% reduction for repetitive JSON with timestamps and field names), encrypts to ~27 KB, and base64-encodes to ~36 KB — well within the 64 KB limit. Compression must happen before encryption (encrypted data is pseudorandom and incompressible). fflate is the recommended compression library: 8 KB bundle (vs pako's 45 KB), fastest in benchmarks, tree-shakeable ESM, DEFLATE/GZIP compatible. `GitHub`

For charts 200 KB–2 MB raw JSON, two paths work: upload the encrypted blob to a Blossom server and store the SHA-256 hash + URL in a standard Nostr event (adds HTTP dependency), or publish to a self-hosted strfry relay configured with `maxEventSize = 2097152`. The self-hosted relay approach requires a single config change and eliminates size concerns entirely, but events only persist on your relay.

Switching from hex to base64 encoding saves ~33% of encoded size (150 KB plaintext: 300 KB hex vs ~200 KB base64). Base64 is valid in Nostr event `content` fields (arbitrary JSON-escaped UTF-8 strings). But encoding alone is insufficient — without compression, even 100 KB base64-encoded content exceeds the 64 KB relay limit.

4. Self-hosted strfry is your persistence guarantee

No public relay guarantees data persistence. Free relays garbage collect old events — they have no obligation to store kind 30078 events forever. For addressable events (kind 30000–39999), relays **MUST** keep only the latest version per coordinate, [\(nostr\)](#) but **MAY** purge even that under storage pressure. Paid relays like **nostr.wine** (~18,888 sats one-time, with regional mirrors) have financial incentives for uptime and retention [\(Nostr\)](#) but still offer no protocol-level guarantee.

The recommended multi-layer architecture:

Your **self-hosted strfry relay** serves as the primary persistence layer — you control retention forever. Add **1–2 paid relays** for funded redundancy, plus **2–3 large public relays** (relay.damus.io, nos.lol) for broad accessibility. [\(Nostr\)](#) Maintain an **IndexedDB client-side cache** as the user's local backup. Use strfry's **negentropy sync** for efficient relay-to-relay backup: `strfry sync wss://backup-relay.com --filter '{"authors":["<pubkey>"]}'`. [\(GitHub\)](#)

NIP-65 (kind 10002 relay list metadata) should be published per user to declare write and read relays. Publish to purplepag.es (specialized for relay list discovery) plus your main relays. The outbox model lets other clients find your users' data. [\(Nostr\)](#) [\(GitHub\)](#)

strfry is the best self-hosted relay for data-centric apps: [\(sourcehut\)](#) C++ with LMDB (embedded, no external database), [\(GitHub\)](#) extremely low resource requirements (1 vCPU, 2 GB RAM, \$5/month VPS), durable writes (never returns OK until committed), websocket compression, and negentropy sync. Configure for 2 MB events:

```
conf

events {
  maxEventSize = 2097152    # 2MB
}

relay {
  maxWebsocketPayloadSize = 2621440 # Must exceed maxEventSize; restart required
}
```

Handling partial publish success is normal in Nostr. Use [\(Promise.allSettled\)](#) to track per-relay results, set a minimum threshold (e.g., 2 of 4 relays must accept), and retry failed relays with exponential backoff. nostr-tools' `pool.publish(relays, event)` returns an array of promises, one per relay.

5. Addressable events replace by timestamp, lowest ID breaks ties

NIP-33 has been **merged into NIP-01** as "addressable events" [\(GitHub\)](#) (the term "parameterized replaceable events" is deprecated). Kind 30078 (NIP-78: application-specific data) is an addressable event [\(GitHub\)](#) [\(GitHub\)](#) identified by the coordinate `30078:<pubkey>:<d-tag>`. **Publishing a new event with the same coordinate and a higher `created_at` replaces the old event on all compliant relays.** [\(nostr\)](#) This is confirmed, **MUST-level** behavior.

Conflict resolution is deterministic. When two events share the same `created_at`, the spec states: "the event with the **lowest id** (first in lexical order) should be retained." `Nostr` `nostr` Event IDs are SHA-256 hashes, so this is unambiguous. However, the spec adds: "these are just conventions and relay implementations may differ" `GitHub` — so the tiebreaker is strong convention, not absolute guarantee.

Split-brain across relays is real. If Device A publishes to Relay 1 and Device B publishes to Relay 2 near-simultaneously, each relay has a different version. Nostr relays do not gossip with each other `Nostr` — events only arrive via client push. Mitigation: publish to the same relay set from all devices (via NIP-65), query before publishing to detect the latest version, and implement client-side reconciliation using the same highest-`created_at`/lowest-ID rule.

Querying is straightforward. Fetch all charts: `{kinds: [30078], authors: [pubkey]}`. Fetch specific chart: add `"#d": ["chart-id"]`. Incremental sync: add `"since": lastSeenTimestamp`. Performance with 50–200 events is fine — relays index by kind and author efficiently. **No prefix or pattern matching** on d-tag values exists in the protocol; only exact match via `#d` filter. To filter by app-specific prefix, fetch all events and filter client-side.

typescript

```
// Deduplication for addressable events
function dedup(events) {
  const map = new Map();
  for (const e of events) {
    const d = e.tags.find(t => t[0] === 'd')[1] || '';
    const key = `${e.kind}:${e.pubkey}:${d}`;
    const existing = map.get(key);
    if (!existing || e.created_at > existing.created_at ||
        (e.created_at === existing.created_at && e.id < existing.id)) {
      map.set(key, e);
    }
  }
  return [...map.values()];
}
```

6. Real-time push works but expect 2–4x Socket.IO latency

NIP-01 specifies that relays **SHOULD** push new matching events to open subscriptions in **real-time**. After delivering stored events and an `EOSE` marker, the relay forwards any newly received events matching the filter for the subscription's lifetime. `nostr` `Stacker News` The `limit` property "is only valid for the initial query and **MUST** be ignored afterwards." `GitHub +2` This is **SHOULD**-level, but virtually all production relays implement it — a relay that doesn't would be considered non-compliant.

Latency is typically 100–300 ms for well-connected relays (same continent), compared to ~50 ms for Socket.IO over your own server. The increase comes from geographic distance to public relays, relay processing overhead, and potential congestion. A self-hosted relay co-located with users narrows this gap significantly. High-traffic public relays can suffer from overload, causing slower propagation. (Nostr)

Reconnection requires re-sending REQ subscriptions — subscriptions are per-connection and lost on disconnect. SimplePool with `enableReconnect: true` handles this automatically, including updating `since:` filters to avoid duplicates. Combine with `enablePing: true` for WebSocket keepalive. (npm) (GitHub) The browser WebSocket API does not expose ping/pong control frames to JavaScript, so SimplePool implements application-level ping internally.

Multiple filters in a single REQ are supported and OR'd together. (E2Encrypted) Combine own charts + shared charts in one subscription:

```
json

["REQ", "charts",
  {"kinds": [30078], "authors": ["<my_pubkey>"]},
  {"kinds": [30079], "#p": ["<my_pubkey>"]}
]
```

Practical limit: ≤ 10 filters per REQ, ≤ 10 kinds per filter. (Nostr) Some relays limit concurrent subscriptions (relay.nostr.band allows ~8). (GitHub)

Detecting stale connections requires application-level monitoring since browser JS can't access WebSocket pings. Use `visibilitychange` and `online` events to probe connections when the tab becomes visible or network returns. SimplePool's built-in ping handles most cases.

7. Two sharing models cover all cases

For recipients with Nostr identity, use NIP-44 encryption (Option B). Encrypt the chart's AES-256-GCM key to the recipient's secp256k1 pubkey using NIP-44's ECDH + HKDF + ChaCha20 construction. (GitHub) (Nostr) The recipient decrypts with their private key — no URL or out-of-band secret needed. For maximum metadata protection, wrap in NIP-59 Gift Wrap (3-layer encryption: unsigned rumor → sealed with author's key → wrapped with ephemeral key), which hides the sender's identity from relays. (Learnnostr) (Nostr)

```
typescript

import { getConversationKey, encrypt } from 'nostr-tools/nip44';
const convKey = getConversationKey(authorSK, recipientPK);
const encryptedAESKey = encrypt(hexEncode(chartAESKey), convKey);
// Store in kind 30079 event with ["p", recipientPK] tag
```


For anonymous recipients (no Nostr identity), use URL fragment secrets (Option A).

Generate a random 256-bit secret, derive a share key via HKDF, encrypt the chart's AES key with the share key, publish as a kind 30079 event, and share the URL `[app.com/share/<event-id>#<secret>]`. The fragment never leaves the browser (not sent in HTTP requests or Referer headers). (Romain-clement) This pattern has strong precedent: Excalidraw, Firefox Send, and Yopass all use it. (Romain-clement) Risk: browser history, clipboard managers, and third-party scripts can access `[window.location.hash]`. Mitigate with `[history.replaceState()]` to strip the fragment after loading.

Collaborative editing faces a fundamental constraint: only the private key holder can publish or update an addressable event under their pubkey. (GitHub) The recommended model for this app is an "edit proposal" pattern — collaborators publish proposal events referencing the original chart's `[a]` tag, and the author's client merges proposals and publishes the update. Full CRDT-based collaboration has no finalized NIP. (Nostr)

Share link expiry uses four layers: NIP-40 `[["expiration", "<timestamp>"]]` tag (relays SHOULD stop serving, MAY not delete), (GitHub) replaceable event tombstone (publish empty content to overwrite), client-enforced expiry timestamp in encrypted content, and NIP-09 deletion event. NIP-40 is explicitly not a security feature — "relays may persist indefinitely" (Nostr) — so layer all four for defense in depth.

8. Disposable keypair handshake is the cleanest device transfer

The bootstrap problem: NIP-44 DMs require the recipient's pubkey, but the new device doesn't have a keypair yet (the mnemonic is what needs transferring). The solution is a **disposable keypair handshake**, which follows the same pattern as NIP-46 (Nostr Connect) and NIP-47 (Nostr Wallet Connect):

1. New device generates a throwaway keypair and displays the pubkey as a QR code
2. Old device scans QR, encrypts mnemonic to throwaway pubkey via NIP-44
3. Old device publishes as a NIP-59 gift-wrapped event with NIP-40 expiration (5 minutes)
4. New device subscribes for kind 1059 events with its throwaway pubkey, receives and decrypts
5. New device derives the real keypair from the mnemonic via NIP-06, discards throwaway key

This is the most secure option because the QR contains only a public key (safe to briefly expose), NIP-44 encryption means only the throwaway private key holder can decrypt, and gift wrapping hides the sender's identity. (GitHub) The pattern has strong ecosystem precedent in NIP-46 and NIP-47.

Ephemeral events (kind 20000–29999) are unreliable for transfer (nostr) — both devices must be connected near-simultaneously, and relays may drop the event before the new device connects. (Stacker News) Use a regular kind with NIP-40 expiration + NIP-09 deletion after transfer completion instead.

The QR + encrypted event fallback (event ID + AES secret in QR, encrypted blob on relay) also works: relay may persist the event, but content is AES-256-GCM encrypted and the decryption secret never touches the relay. Brute-forcing 256-bit AES is infeasible, making persistence acceptable from a security standpoint.

9. Deletion is a request, not a guarantee

NIP-09 defines "deletion requests" (kind 5 events), not deletions. The spec deliberately uses SHOULD, not MUST: relays SHOULD delete referenced events but are not obligated to. [GitHub](#)

[Nostr](#) The spec warns: "it is impossible to delete events from all relays and clients." [Nostr](#)

Most major relay implementations (strfry, nostr-rs-relay) do honor NIP-09, [sourcehut](#) and relays advertise support via NIP-11's [supported_nips](#) field. [E2Encrypted](#)

For addressable events, use the [a](#) tag (not just [e](#)) to reference the stable coordinate [30078:<pubkey>:<d-tag>](#). This covers all versions: [GitHub](#)

typescript

```
const deletion = finalizeEvent({
  kind: 5,
  created_at: Math.floor(Date.now() / 1000),
  tags: [
    ['a', `30078:${pubkey}:${dTag}`],
    ['k', '30078']
  ],
  content: 'removed'
}, sk);
```

The most reliable "deletion" for addressable events is a soft delete — publish a replacement with empty content. This leverages the MUST-level replaceable event behavior in NIP-01, [Nostr](#) meaning every compliant relay will store the empty tombstone in place of the original content. Combine both approaches: soft delete first (guaranteed replacement), then NIP-09 deletion request (best-effort full removal). Since chart content is encrypted, even if both approaches fail on some relay, the persisted ciphertext is unreadable without the key.

10. Queue by coordinate, publish only the final state

Per-relay publish tracking with IndexedDB persistence is the proven pattern. Each queue entry stores the signed event plus a [relayStatus](#) map tracking acceptance status, attempt count, and last attempt timestamp per relay. On [online](#) events and [visibilitychange](#), retry failed relays with exponential backoff.

For NIP-33 addressable events, deduplicate the queue by coordinate ([kind:pubkey:d-tag](#)). If a user edits the same chart 5 times offline, only the final version needs publishing. Key the IndexedDB store by coordinate and overwrite with the latest [created_at](#) version:

typescript

```
async function queueReplaceable(signedEvent, relays) {
  const dTag = signedEvent.tags.find(t => t[0] === 'd')?.[1] || '';
  const coordinate = `${signedEvent.kind}:${signedEvent.pubkey}:${dTag}`;
  const existing = await db.get('queue', coordinate);
  if (!existing || signedEvent.created_at >= existing.event.created_at) {
    await db.put('queue', { coordinate, event: signedEvent, relayStatus: /*...*/ });
  }
}
```

Timestamp conflicts with stale queued events: if the offline queue holds an event with `created_at: 1000` and the relay already has `created_at: 1500` from another device, the relay will likely accept the older event (valid signature) but will not replace the newer one. The newer event wins. This means **offline changes can be silently lost** under the last-write-wins model. Mitigation: before publishing queued events, fetch the latest version from relays, detect conflicts, and either merge application-level content or prompt the user. Re-sign with the current timestamp to avoid NIP-22 rejection (some relays reject events with `created_at` too far in the past). (Nexta)

11. Re-encrypt everything during a 3–6 month dual-write period

Dual-write is the safest migration path: write each chart update to both Flask and Nostr, read from both, prefer the newer timestamp. The Nostr side uses kind 30078 with the same d-tag as the chart identifier. (AsaiToshiya on Nostr) Maintain dual-write for 3–6 months until confident in Nostr persistence, then decommission Flask. Complexity is moderate — the main challenge is maintaining two write paths and reconciling timestamps across systems.

Bulk migration: publish existing charts as kind 30078 events sequentially with 500 ms–1 s delays between events. Most relays tolerate 10–100 events per minute. For hundreds of charts, use your own relay (no rate limits) as primary and publish to public relays in slower batches. Monitor `[["OK", <id>, true/false, <message>]]` responses for rejections. (Nostr)

The P-256 → secp256k1 curve change is a clean break. The same BIP39 mnemonic produces entirely different keys on each curve (different derivation paths, different curve equations, different HMAC seeds in BIP32 vs SLIP-0010). Old P-256 ECDSA signatures are mathematically unverifiable with secp256k1 keys. The recommended strategy: **stop verifying old signatures, re-encrypt chart data with keys derived from the new secp256k1 hierarchy, sign all new events with Schnorr.** Keep P-256 verification code in the app as read-only legacy support during the dual-write period, then remove it.

Identity continuity requires an app-internal mapping since the user's pubkey changes. Use **NIP-05 DNS verification** (`user@yourdomain.com` → new Nostr pubkey) for human-readable identity continuity. Publish a cross-signed migration announcement: sign the new secp256k1 pubkey with the old P-256 key (stored on Flask), and publish a Nostr event from the new identity referencing the old. No finalized NIP for key migration exists — NIP-41 and PR #1452 remain in draft. [GitHub](#) [GitHub](#) For a single-purpose app, internal identity mapping is simpler and more reliable than social-graph-based migration.

12. Encrypt content, hash d-tags, verify signatures client-side

Chart UUIDs in d-tags leak metadata. An observer can count events, track update frequency, and correlate sharing relationships. **Hash all UUIDs:** `d_tag = SHA256(chart_uuid || HKDF(private_key, "dtag-salt"))`. The salt derived from the user's key prevents external correlation while keeping lookups deterministic for the user's own client. The number of events and timing patterns remain visible regardless — accept this or use AUTH-protected relays.

Encrypted blob size reveals chart complexity. For educational chart data this is low risk; for medical data it could indicate condition severity or number of medications. NIP-44 includes power-of-two padding but acknowledges "limited message size leak." [Nostr](#) Implement **application-level padding to fixed size buckets** (8 KB, 32 KB, 128 KB) before encryption, independently of NIP-44 padding. For maximum privacy with sensitive data, pad all charts to a uniform maximum size.

Social graph leakage via p-tags is the most significant metadata risk. Relays see which pubkeys share with whom when p-tags are used in share events. [GitHub](#) **NIP-59 Gift Wrap** mitigates this: the outer event is signed by a random ephemeral key (hiding the sender), and NIP-42 AUTH restricts who can query kind 1059 events (partially hiding the recipient). [E2Encrypted +2](#) The relay still sees the recipient pubkey in the p-tag, but cannot determine the sender. Use AUTH-enabled relays for sharing.

Relay-side Schnorr signature verification is common but not guaranteed. NIP-01's security model is "relays don't have to be trusted — signatures are verified on the client side." [GitHub](#) [GitHub](#) Major implementations (strfry, nostr-rs-relay, Nosflare) do verify server-side, but a 2025 EuroS&P paper found several popular clients omitted verification, enabling event forgery and profile impersonation. [Crypto-sec-n](#) **Always verify every event's Schnorr signature client-side** before processing — recompute the event ID from the payload and verify the signature against the claimed pubkey.

NIP-44 DMs (without Gift Wrap) leak sender and recipient pubkeys — both are visible in the event and p-tag. [GitHub](#) NIP-04 is even worse (deprecated, uses broken AES-CBC with unhashed ECDH). [GitHub](#) [+3](#) **NIP-44 + NIP-59 together** hide the sender identity, true event kind, real timestamps, and all inner tags. The recipient pubkey remains visible to the relay in the gift wrap's p-tag, mitigated by NIP-42 AUTH. For this app, use NIP-44 for self-encrypted storage (kind 30078) and NIP-44 + NIP-59 for any inter-user sharing.

Conclusion

The migration's feasibility rests on three architectural decisions. First, **compress before encrypt** using `fflate` + `base64`, which brings most charts under the 64 KB public relay limit. Second, **self-host a strfry relay** configured for 2 MB events as your persistence backstop — this is a single config change on a \$5/month VPS. Third, adopt **Blossom blob storage** as the escape valve for charts that exceed relay limits even after compression.

The JavaScript cryptography stack is mature and performant: `@noble/curves` for Schnorr signing (~1–3 ms), `npm` Web Crypto for AES-256-GCM (native speed), and `@scure/bip39` + `@scure/bip32` for NIP-06 key derivation. `nostr-tools` v2.23.0 provides everything else `npm` at ~20–25 KB gzipped. The biggest novel insight from this research is that **soft delete (publishing empty content) is more reliable than NIP-09 deletion** for addressable events, because replaceable event behavior is MUST-level spec `Nostr` while deletion is merely SHOULD. `Nostr` The most underappreciated risk is not blob size or performance — it's the **last-write-wins conflict model** when editing from multiple devices while offline, which requires application-level merge logic that Nostr's protocol does not provide.